1. INTRODUCTION

1.1 Project Overview

The Stock Trading Simulation Web Application is a web-based platform designed to help users practice stock trading without using real money. The system simulates key features of modern trading platforms such as stock browsing, buying and selling shares, portfolio tracking, and transaction history.

The application is built using the MERN stack, which includes **MongoDB for database management, Express.js and Node.js for backend development, and React for the frontend interface.

Users can register, log in securely, view available stocks, and perform simulated buy or sell operations using virtual currency. The portfolio updates automatically after each transaction, allowing users to track their performance through an interactive dashboard.

This platform serves as an educational tool that helps beginners understand stock market operations and trading strategies without financial risk.

1.2 Purpose

The purpose of this project is to create an interactive learning environment where users can gain practical experience in stock trading. Many beginners are interested in investing but hesitate due to the financial risks involved.

This application provides a safe environment where users can experiment with trading strategies, monitor portfolio performance, and understand stock market behavior. The project also demonstrates the implementation of modern web technologies and full-stack application development.

---

2. IDEATION PHASE

2.1 Problem Statement

Stock trading has gained popularity with the growth of digital investment platforms. However, beginners often avoid investing due to the fear of financial loss and lack of practical experience.

Most existing trading platforms require real investments, making them unsuitable for learning purposes. Educational resources mainly focus on theoretical knowledge rather than practical experience.

Therefore, a simulation platform is required that allows users to practice trading in a risk-free environment. The proposed Stock Trading Simulation Application provides such a

platform where users can learn trading concepts and test strategies without financial consequences.

2.2 Empathy Map Canvas

Target Users

- Beginner investors
- Students interested in finance or technology
- Individuals wanting to learn stock trading

User Thoughts and Feelings
Users feel uncertain about investing due to lack of knowledge and experience.

User Observations
Many trading platforms appear complex and difficult for beginners to understand.

User Needs

- Easy-to-use trading interface
- Risk-free practice environment
- Understanding of market trends

Pain Points

- Fear of losing money
- Lack of real trading experience
- Difficulty analyzing market data

User Gains

- Practical trading knowledge
- Confidence in financial decision-making
- Ability to test investment strategies

2.3 Brainstorming

During the ideation stage, several fintech project ideas were considered, including financial dashboards, stock prediction systems, and investment tracking tools.

After evaluation, the Stock Trading Simulation platform was selected because it combines financial education with practical experience. The brainstorming process focused on designing features such as user authentication, stock listings, portfolio management, and transaction tracking while ensuring a simple and user-friendly interface.

---

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey outlines how users interact with the application:

1. **User visits the application website.**
2. **User registers and creates an account.**
3. **User logs into the system.**
4. **User accesses the dashboard and views available stocks.**
5. **User searches for specific stocks.**
6. **User performs buy or sell operations using virtual funds.**
7. **Portfolio updates automatically after each transaction.**
8. **User reviews transaction history and portfolio performance.**

This process ensures a smooth and engaging user experience.

3.2 Solution Requirements

Functional Requirements

The system must provide the following features:

- **User registration and login**
- **Secure authentication**
- **Stock listing and search functionality**
- **Buy and sell stock simulation**
- **Portfolio management**
- **Transaction history tracking**
- **Watchlist feature**
- **Admin panel for stock management**

Non-Functional Requirements

The system must also ensure:

- **Security: Protection of user data through encrypted authentication**
- **Performance: Fast transaction processing**
- **Usability: Simple and intuitive interface**
- **Scalability: Support for multiple users**
- **Reliability: Accurate storage of transactions and portfolio data**

3.3 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how information moves through the system.

Users interact with the web interface to perform operations such as login, stock browsing, and trading. These requests are sent to the backend server, which processes them and communicates with the database.

The database stores information related to users, stocks, portfolios, and transactions. The processed data is then returned to the frontend interface and displayed to the user.

3.4 Technology Stack

Frontend

The frontend is developed using React, which allows the creation of dynamic and responsive user interfaces through reusable components.

Backend

The backend is developed using Node.js with Express.js, enabling efficient API development and communication with the database.

Database

The application uses MongoDB, a NoSQL database that stores data in flexible document structures.

Development Tools

- **Postman for API testing**
- **GitHub for version control and collaboration**

---

4. PROJECT DESIGN

4.1 Problem Solution Fit

The proposed system addresses the problem of financial risk in stock trading education. By offering a simulation platform, users can learn trading concepts and experiment with strategies without risking real money.

4.2 Proposed Solution

The application provides a virtual trading platform where users can register, browse stocks, and perform simulated buy or sell operations.

Each transaction automatically updates the user's portfolio and transaction history. Administrators can manage stock listings through a dedicated admin panel.

4.3 Solution Architecture

The system follows a three-tier architecture consisting of:

1. **Presentation Layer: React frontend interface**
2. **Application Layer: Node.js and Express backend server**
3. **Data Layer: MongoDB database**

This architecture enables efficient communication between the user interface, business logic, and data storage.

---

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was developed through several stages:

1. **Requirement analysis**
2. **System design and UI planning**
3. **Backend API development**
4. **Frontend implementation**
5. **Integration of frontend and backend**
6. **Testing and debugging**

7. **Final deployment and documentation**

The development process was completed over several weeks with continuous testing and improvements.

---

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Testing

Testing was performed to ensure system reliability and performance.

Functional Testing

- **User registration and login**
- **Stock search functionality**
- **Buy and sell transactions**
- **Portfolio updates**
- **Transaction history tracking**

Performance Testing

- **API response time**
- **Database query performance**
- **System responsiveness under multiple requests**

The testing results confirmed that the system operates efficiently and reliably.

---

7. RESULTS

7.1 Output Screenshots

The application successfully implemented all core features. Screenshots included in the report may show:

- **Login page**
- **Registration page**
- **User dashboard**
- **Stock listing page**
- **Portfolio page**

- Transaction history page

These outputs demonstrate the functionality and usability of the application.

---

8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides risk-free trading practice
- Helps beginners understand stock market concepts
- User-friendly and responsive interface
- Portfolio tracking and transaction history

Disadvantages

- No real financial transactions
- Stock data may not be real-time
- Requires internet connectivity

---

9. CONCLUSION

The Stock Trading Simulation Web Application demonstrates the development of a full-stack web application using the MERN stack. The platform provides a safe environment where users can practice stock trading and gain practical knowledge of financial markets.

The project highlights the importance of combining financial education with modern web technologies to create interactive learning platforms.

---

10. FUTURE SCOPE

Future improvements may include:

- Integration with real-time stock market APIs
- AI-based stock prediction and analytics
- Mobile application development
- Social trading features
- Advanced portfolio analytics

---

11. APPENDIX

Source Code

Project source code hosted on
GitHub repository.

Dataset / API Link

Stock market data sources used for the application.

Project Links

- **GitHub repository**
- **Application demo video**

---

12. SYSTEM ARCHITECTURE

The application follows a three-tier architecture consisting of frontend, backend, and database layers. The frontend built with React handles user interaction, the backend developed using Node.js and Express processes requests, and MongoDB stores application data.

---

13. DATABASE SCHEMA DESIGN

The database contains several collections:

Users Collection
Stores user details such as name, email, encrypted password, and account balance.

Stocks Collection
Contains stock information including company name, symbol, and price.

Portfolio Collection
Maintains records of stocks owned by users.

Transactions Collection
Stores details of buy and sell operations.

---

14. ER DIAGRAM EXPLANATION

The system consists of four main entities: User, Stock, Portfolio, and Transaction.

- **A user can perform multiple transactions.**
- **A user can own multiple stocks in their portfolio.**
- **A stock can appear in many portfolios and transactions.**

This structure ensures efficient management of trading data.

---

15. SECURITY IMPLEMENTATION

Security measures implemented in the system include:

- **Password hashing to protect user credentials**
- **JWT-based authentication for secure login**
- **Role-based access control for admin features**
- **Input validation to prevent malicious data**

---

16. USER INTERFACE DESIGN

The application interface focuses on simplicity and usability.

Key UI features include:

- **Responsive dashboard layout**
- **Easy navigation between pages**
- **Interactive stock tables**
- **Buy and sell trading windows**
- **Portfolio performance summary**

The responsive design ensures compatibility across desktop and mobile devices.